

Faculty of Computing and Information Technology (FCIT)

Machine Learning **ASSIGNMENT # 2**

Instructor: Dr. Madiha Khalid

BS(SE)-Fall 2023

Total Marks: **100**Deadline: **Mar 17, 2026 11:59 PM**

Multiple Linear Regression & Logistic Regression

Instructions

- **Work must be your own.** No copy, no plagiarism (0 marks if found).
- Use libraries like **numpy, pandas, matplotlib, seaborn** only for data handling and plotting.
- **Do not** use built-in regression or classification models (e.g., **sklearn.linear_model**).
- Implement all regression and classification functions **from scratch** using NumPy.
- Submit as a **Colab/Jupyter Notebook** with clear section headings and comments.

Objective

This assignment covers two fundamental supervised learning paradigms. You will implement Multiple Linear Regression (MLR) to predict a continuous target variable from several features, and Logistic Regression to solve a binary classification problem. Both algorithms will be built entirely from scratch using NumPy and evaluated with appropriate metrics.

Tasks Overview

Part	Topic	Marks
1	Dataset Load and Preprocessing	25
2	Multiple Linear Regression	30
3	Logistic Regression (Classification)	30
4	Comparison & Visualization	15

Background Theory

Multiple Linear Regression (MLR)

Multiple Linear Regression generalizes simple linear regression to handle more than one predictor variable. Instead of fitting a line in 2-D, it fits a hyperplane in n-dimensional space.

Hypothesis Function:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n \quad (\text{or in matrix form: } h(X) = X \cdot \theta)$$

where:

- $\theta_0 \rightarrow$ bias/intercept term
- $\theta_1 \dots \theta_n \rightarrow$ weights for each feature $x_1 \dots x_n$
- $X \rightarrow$ design matrix ($m \times (n+1)$), with a leading column of 1s for the bias

Logistic Regression

Logistic Regression is a supervised learning algorithm for binary classification. Rather than predicting a continuous value, it models the probability that an input belongs to class 1. The key idea is to pass a linear combination of features through the sigmoid function to squash its output into the range (0, 1).

Sigmoid (Hypothesis) Function:

$$h(x) = \sigma(z) = 1 / (1 + e^{(-z)}) \quad \text{where } z = X\theta = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

The output $h(x)$ is the predicted probability. A decision threshold (typically 0.5) converts it to a class label: predict 1 if $h(x) \geq 0.5$, else predict 0.

Cost Function — Mean Squared Error (MSE) for MLR

Multiple Linear Regression uses MSE to measure prediction error:

$$J(\theta) = (1 / 2m) \cdot \sum_{i=1}^m (h(x_i) - y_i)^2$$

The factor 1/2 is included for mathematical convenience when computing gradients.

Gradient Descent Update Rule

$$\theta_j := \theta_j - \alpha \cdot (\partial J / \partial \theta_j)$$

$$\text{In matrix form: } \theta := \theta - (\alpha / m) \cdot X^T \cdot (X\theta - y)$$

where α is the learning rate and m is the number of training samples.

Part 1: Dataset Load and Preprocessing

(Marks: 25)

Dataset: Use the file 'Housing.csv'.

You can directly download from Kaggle as follow:

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("yasserh/housing-prices-dataset")

print("Path to dataset files:", path)
```

Dataset Description

The Housing dataset contains information about residential properties such as:

- area
- number of bedrooms
- number of bathrooms
- parking availability
- furnishing status
- and other amenities

The target variable is **price**.

1.1 Load the Dataset (5 marks)

1. **Load** the CSV file using pandas.
2. Print the first 5 rows (**.head()**) and dataset shape.
3. Display basic statistics using **.describe()**.

1.2 Preprocessing (20 marks)

a) Handle Missing Values (5 marks)

- Check for missing/null values in each column.
- Fill missing numeric values with the **column mean**; drop rows only if > 30% of values are missing.

b) Feature Scaling (8 marks)

- Apply **Z-score Standardization** (mean = 0, std = 1) to all **feature columns (important)**.

$$x_scaled = (x - \mu) / \sigma$$

- Store μ and σ per feature; you will need them later for inverse-transforming predictions.

c) Train / Test Split (7 marks)

- Split the data into **80% training** and **20% testing** sets.
- Do **not** use sklearn's **train_test_split**; implement the split manually using array indexing or random shuffling.
- Report the number of samples in each split.

Part 2: Multiple Linear Regression

(Marks: 30)

Implement Multiple Linear Regression from scratch. You must implement all functions below without using any built-in regression libraries.

2.1 Design Matrix (5 marks)

Add a column of **ones** as the first column of X to account for the bias term θ_0 .

$X_b = [1 \ x_1 \ x_2 \ \dots \ x_n]$ (each row is one training sample)

2.2 Hypothesis Function (5 marks)

Implement the vectorized hypothesis.

2.3 Cost Function (8 marks)

Implement MSE cost function.

2.4 Gradient Descent (12 marks)

Implement the gradient descent.

- **Run** gradient descent on the training set.
- Report the final **θ values** and the **minimized training cost**.
- Compute predictions on the test set and report **Test MSE**.
- Learning rate $\alpha = 0.01$
- Iterations = **1000**

Part 3: Logistic Regression (Classification)

(Marks: 30)

Using the same Housing dataset, create a binary classification target by labelling each house as **1 (expensive)** if its price is above the median, and **0 (affordable)** otherwise. Implement Logistic Regression from scratch to classify houses into these two categories (**important**).

3.1 Dataset Preparation for Classification (5 marks)

- Create a binary target column: assign $y = 1$ if price > median price, else $y = 0$.
- Encode any remaining categorical features (e.g., **furnishingstatus**) using label encoding, then apply feature scaling using the same mean-normalisation approach as Part 1.
- Use a train-test split (consistent with Part 1).

3.2 Sigmoid Function (5 marks)

- Implement a function **sigmoid(z)** that computes $\sigma(z) = 1 / (1 + e^{-z})$ element-wise using NumPy. This will serve as the hypothesis function: $h(x) = \sigma(X\theta)$.
- Verify your implementation by testing $\text{sigmoid}(0) = 0.5$, $\text{sigmoid}(-100) \approx 0$, and $\text{sigmoid}(100) \approx 1$.

3.3 Cost Function — Binary Cross-Entropy (8 marks)

- Implement the binary cross-entropy cost function from scratch:
$$J(\theta) = -(1/m) \cdot \sum [y \cdot \log(h) + (1-y) \cdot \log(1-h)]$$
- Add a small epsilon (e.g., $1e-8$) inside the logarithms to prevent **log(0)** numerical errors.
- Record and plot the cost after every iteration.

3.4 Gradient Descent for Logistic Regression (12 marks)

- Implement gradient descent to minimise $J(\theta)$. The update rule is:

$$\theta := \theta - (\alpha/m) \cdot X^T \cdot (h - y)$$

- Run gradient descent for at least 1000 iterations. Experiment with two different learning rates (e.g., $\alpha = 0.01$ and $\alpha = 0.1$) and report which converges better (**prepare the 'why' behind it**).

3.5 Predictions & Evaluation (5 marks)

- Apply a decision threshold of 0.5 to $h(x)$ to obtain predicted class labels. Compute the **classification accuracy** on the test set.
- Build a **confusion matrix** manually (count TP, TN, FP, FN from predictions) and print it as a 2x2 table.

Hint: Do not use sklearn for the sigmoid, cost, gradient descent, or confusion matrix. Accuracy = (TP + TN) / total samples. All computations must use NumPy only.

Part 4: Comparison & Visualization

(Marks: 15)

Produce the following plots using matplotlib. Label all axes, add a title, and include a legend where appropriate.

4.1 Learning Curves (5 marks)

- Plot the **cost vs. iteration** curve for your MLR model (Part 2).
- The x-axis should be the iteration number; the y-axis the cost value.
- Verify visually that the cost is **decreasing** and converging.

4.2 Logistic Regression Cost Curve (5 marks)

- Plot the **cross-entropy cost vs. iteration** curve for your logistic regression model (Part 3).
- On the same plot, overlay the cost curves for both learning rates you tested ($\alpha = 0.01$ and $\alpha = 0.1$) using different colors. Add a legend.

4.3 Confusion Matrix Heatmap (5 marks)

- Using seaborn or matplotlib, visualise the confusion matrix from Part 3 as a **colour-coded 2x2 heatmap**. Label the axes as Predicted and Actual, and annotate each cell with its count.
- Briefly comment on what the heatmap reveals about your model's classification performance (e.g., which class it predicts better).

Submission Guidelines

- **File format:** Single **.ipynb** (Jupyter / Colab) notebook.
- **Naming convention:** Roll_Number_Name_A2.ipynb (e.g., **BSEF23XXXX_Ali_A2.ipynb**)
- **Comments:** Every function must have a docstring explaining its purpose, inputs, and outputs.
- **No imports:** Do not import sklearn, statsmodels, or any regression utility. NumPy, pandas, matplotlib, seaborn only.
- **Run all cells** before submissionthe, notebook must execute top-to-bottom without errors.

Grading Rubric

Part	Criteria	Max Marks	Score
Part 1	Correct loading, missing value handling, standardization, manual split	25	
Part 2	Design matrix, hypothesis, cost function, gradient descent, test MSE	30	
Part 3	Sigmoid implementation, binary cross-entropy cost, gradient descent update, predictions with threshold, accuracy and confusion matrix (TP/TN/FP/FN)	30	
Part 4	Three required plots: MLR cost curve, LR cost curve (both learning rates), confusion matrix heatmap — all correctly labelled with titles and legends	15	
	Total	100	

Academic Integrity: Submitted work will be checked for plagiarism. All students may be asked to explain their implementation in a viva. Zero tolerance for copying.

Good Luck!

Dr. Madiha Khalid · FCIT · Machine Learning · Spring 2026